NODE2VEC

Node2Vec trong project node2vec

Từ biased random walk đến embedding bằng Skip-gram với negative sampling

Tài liệu kỹ thuật theo mã nguồn hiện tại
Preprocess.py · Node2vecWalk.py · Node2Vec.py · Skip_gram.py

Mục lục và phạm vi

• Toàn cảnh pipeline và ký hiệu
• Tiền xử lý chuyển trạng thái và alias_edges
• Biased random walk
• Tạo positive samples
• Tạo negative samples
• Tạo dataset và batch
• Mô hình Skip-gram
• Loss, lan truyền ngược và cập nhật tham số
• Embedding đầu ra, cấu hình hiện tại và các lưu ý triển khai

Tài liệu mô tả hành vi thực tế của code trong repository. Những điểm khác với Node2Vec/word2vec chuẩn được đánh dấu rõ để tránh nhầm giữa thuật toán lý thuyết và implementation hiện tại.

1. Toàn cảnh pipeline

Node2Vec xem mỗi random walk như một câu, node như một từ, còn các node gần nhau trong walk như các từ nằm trong cùng một cửa sổ ngữ cảnh.

Ký hiệu | Ý nghĩa | Trong code

G | Đồ thị đầu vào | NetworkX graph

p | Điều chỉnh xác suất quay lại node vừa đi qua | self.p

q | Điều chỉnh xu hướng đi ra xa hoặc khám phá gần | self.q

d | Kích thước embedding | dimension

r | Số walk bắt đầu từ mỗi node | walks_per_node

L | Số bước lặp khi tạo một walk | walk_length; danh sách kết quả dài L + 1

w | Context window một chiều trong implementation này | context_size

K | Số negative node cho mỗi positive pair | num_negative

2. Tiền xử lý chuyển trạng thái

2.1. Vì sao cần trạng thái gồm hai node?

Xác suất bước kế tiếp không chỉ phụ thuộc vào node hiện tại v. Nó còn phụ thuộc vào node vừa đi qua t. Vì vậy trạng thái được viết là cặp (t, v):

```
t  →  v  →  x
previous  current  candidate
```

Biết t giúp phân biệt ba trường hợp đối với ứng viên x: quay lại t, đi tới vùng lân cận của t, hoặc đi ra vùng xa hơn.

2.2. Dữ liệu được chuẩn bị

```
neighbors = {
    v: list(G.neighbors(v))
    for v in G.nodes()
}

neighbor_sets = {
    v: set(neighbors[v])
    for v in G.nodes()
}
```

neighbors[v] giữ thứ tự các node kề với v. neighbor_sets[v] giúp kiểm tra x in N(t) nhanh hơn khi phân loại ứng viên.

2.3. Công thức trọng số Node2Vec

Với mỗi cạnh vô hướng (u,v), code tạo hai trạng thái có hướng: (u,v) và (v,u). Với trạng thái (t, curr), xét mọi x ∈ N(curr):

$$\alpha(t, x) = 1/p \text{ nếu } x = t;\ 1 \text{ nếu } x \in N(t);\ 1/q \text{ nếu còn lại}$$

Trọng số chuyển tiếp là:

$$\pi_{curr,x} = w(curr,x) \times \alpha(t,x)$$

Sau đó chuẩn hóa:

$$P(x \mid t, curr) = \pi_{curr,x} / \Sigma_{y \in N(curr)} \pi_{curr,y}$$

Trường hợp | Hệ số | Ý nghĩa

x == t | 1/p | Quay lại node trước đó. p lớn làm giảm xác suất quay lại; p nhỏ làm tăng.

x in N(t) | 1 | Đi tới node gần node trước đó, vẫn ở vùng khám phá hiện tại.

Trường hợp còn lại | 1/q | Đi ra xa vùng hiện tại. q nhỏ khuyến khích exploration; q lớn khuyến khích BFS-like.

2.4. alias_edges trong implementation này

```
alias_edges[(t, curr)] = {
    "candidates": candidates,
    "probs": probs
}
```

Tên biến là alias_edges, nhưng code hiện tại chưa xây dựng bảng Vose alias gồm hai mảng J và q. Nó thực hiện cách đơn giản hơn: tính trước danh sách ứng viên và vector xác suất chuẩn hóa, rồi dùng trực tiếp với np.random.choice(..., p=probs). Kết quả phân phối là đúng, nhưng tên “alias” ở đây mang nghĩa cache transition probabilities.

3. Tạo một biased random walk

3.1. Bước đầu tiên

Hàm random_walk(G, start, walk_length, alias_edges) bắt đầu bằng:

```
walk = [start]
```

Ở bước đầu tiên, chưa có node trước đó nên không thể dùng trạng thái (t, curr). Code dùng trọng số cạnh gốc:

```
candidates = list(G.neighbors(current))
weights = [G[current][x].get("weight", 1.0) for x in candidates]
probs = weights / sum(weights)
next_node = np.random.choice(candidates, p=probs)
```

Nếu cạnh không có thuộc tính weight, trọng số mặc định là 1.0.

3.2. Các bước tiếp theo

```
previous = walk[-2]
current = walk[-1]
trans = alias_edges[(previous, current)]
candidates = list(trans["candidates"])
probs = trans["probs"]
next_node = np.random.choice(candidates, p=probs)
walk.append(next_node)
```

Mỗi bước dùng hai node cuối hiện có để tra đúng phân phối đã tiền tính. Vì vậy Node2Vec là random walk bậc hai: xác suất bước kế tiếp phụ thuộc vào node hiện tại và node trước đó.

3.3. Ví dụ

Giả sử tại trạng thái A → B, các ứng viên của B là A, C, D. Nếu A là node trước, C kề với A, còn D không kề với A, thì:

```
base weights       : w(B,A), w(B,C), w(B,D)
alpha              : 1/p,   1,      1/q
modified weights   : w(B,A)/p, w(B,C), w(B,D)/q
probabilities      : modified weights / their sum
```

Node được chọn ngẫu nhiên theo vector xác suất này và được append vào walk.

3.4. Số lượng walk và độ dài

```
for i in range(self.walks_per_node):
    for node in self.G.nodes():
        walks.append(random_walk(...))
```

Nếu đồ thị có |V| node thì tổng số walk là |V| × walks_per_node. Do code khởi tạo walk bằng node bắt đầu rồi lặp range(walk_length), kết quả có walk_length + 1 node. Đây là một chi tiết khác với cách một số thư viện gọi walk_length là tổng số node.

4. Tạo positive samples

Hàm positive_samples biến mỗi walk thành các cặp Skip-gram. Trong implementation hiện tại, mỗi vị trí lần lượt làm source, và chỉ lấy tối đa context_size node ở phía sau.

```
for walk in walks:
    for i, source in enumerate(walk):
        end = min(i + context_size + 1, len(walk))
        contexts = walk[i + 1:end]
        for context in contexts:
            positive_samples.append((source, context))
```

enumerate(walk) sinh từng cặp (i, walk[i]). Vì vậy i là vị trí và source là node ở vị trí đó.

Ví dụ với walk = [A, B, C, D], context_size = 2

i | source | contexts | positive pairs

0 | A | [B, C] | (A,B), (A,C)

1 | B | [C, D] | (B,C), (B,D)

2 | C | [D] | (C,D)

3 | D | [] | không có

Các cặp lặp lại không bị loại. Đây thường là chủ ý: tần suất đồng xuất hiện trong các walk được giữ lại như một tín hiệu huấn luyện.

5. Tạo negative samples

5.1. Ý tưởng

Với positive pair (source, positive_context), ta lấy thêm K node ngẫu nhiên làm negative context. Mục tiêu của mô hình là tăng điểm cho positive pair và giảm điểm cho các negative pair.

```
(source, positive_context) → label 1
(source, negative_1)       → label 0
(source, negative_2)       → label 0
...
(source, negative_K)       → label 0
```

5.2. Phân phối noise

```
node_counts = Counter(
    node for walk in walks for node in walk
)

frequencies = [node_counts[node] ** 0.75 for node in nodes]
probs = frequencies / frequencies.sum()
```

Node xuất hiện nhiều trong corpus có xác suất được lấy cao hơn, nhưng lũy thừa 0.75 làm phân phối bớt lệch so với tần suất gốc. Đây là phân phối noise kinh điển của negative sampling.

5.3. Loại các node không hợp lệ và lấy mẫu

```
valid_mask = [
    node != source and node != positive_context
    for node in nodes
]

candidates = nodes[valid_mask]
candidate_probs = probs[valid_mask]
candidate_probs /= candidate_probs.sum()

negatives = np.random.choice(
    candidates,
    size=num_negative,
    replace=True,
    p=candidate_probs
)
```

Source và positive context hiện tại bị loại. replace=True cho phép một node negative xuất hiện nhiều lần trong cùng một nhóm, ví dụ [C, D, C, E, C]. Vì vậy số negative có thể đếm đúng là K nhưng không nhất thiết có K node khác nhau.

6. Tạo dataset và batch

6.1. Ánh xạ node sang integer ID

```
node_ids = list(G.nodes())
node2id = {node: idx for idx, node in enumerate(node_ids)}
```

nn.Embedding cần chỉ số nguyên từ 0 tới |V|-1. Mapping giữ đúng thứ tự G.nodes(), và thứ tự này cũng là thứ tự các dòng embedding khi trả kết quả.

6.2. Các tensor

Gọi N là số positive pairs và K là số negative cho mỗi pair:

Tensor | Shape | Nội dung

source_ids | (N,) | ID source của từng positive pair

positive_context_ids | (N,) | ID context dương tương ứng

negative_context_ids | (N,K) | K ID negative được xếp theo từng positive pair

Negative pairs được tạo thành một list phẳng theo đúng thứ tự mỗi positive pair. Trong fit(), đoạn slice:

```
negative_pairs[i*K : (i+1)*K]
```

đưa chúng về hàng thứ i của tensor (N,K).

6.3. TensorDataset và DataLoader

```
dataset = TensorDataset(
    source_ids,
    positive_context_ids,
    negative_context_ids,
)
train_loader = DataLoader(
    dataset,
    batch_size=batch_size,
    shuffle=True,
    drop_last=False,
)
```

Mỗi item của dataset là một bộ ba:

```
(source_id, positive_context_id, negative_context_ids[K])
```

DataLoader trộn thứ tự các item ở mỗi epoch và gom thành batch. drop_last=False giữ lại batch cuối dù nhỏ hơn batch_size.

7. Mô hình Skip-gram

7.1. Hai bảng embedding

```
self.source_embedding = nn.Embedding(num_nodes, embedding_dim)
self.context_embedding = nn.Embedding(num_nodes, embedding_dim)
```

Với |V| node và kích thước d, mỗi bảng có shape (|V|, d):

• U: source embeddings, một vector u_i cho mỗi node.

• V: context embeddings, một vector v_j cho mỗi node.

Khởi tạo trong code:

```
source_embedding ~ Uniform(-0.5/d, 0.5/d)
context_embedding = 0
```

Hai bảng tách biệt giúp mô hình biểu diễn vai trò source và context khác nhau trong quá trình tối ưu.

7.2. Forward bằng dot product

```
source_vectors = source_embedding(sources)
context_vectors = context_embedding(contexts)
logits = (source_vectors * context_vectors).sum(dim=-1)
```

Với một cặp (i,j):

$$s(i,j) = u_i^T v_j$$

Dot product càng lớn thì cặp source-context càng được mô hình xem là phù hợp. Với positive batch, output có shape (B,). Với negative batch, code truyền sources.unsqueeze(1) để source có shape (B,1), broadcast với context shape (B,K), nên output có shape (B,K).

8. Loss với negative sampling

Gọi s+ là logit của positive context và sk- là logit của negative thứ k. Code dùng:

```
positive_loss = -F.logsigmoid(positive_logits)
negative_loss = -F.logsigmoid(-negative_logits).sum(dim=1)
loss = (positive_loss + negative_loss).mean()
```

$$L = -\log \sigma(s+) - \sum_{k=1..K} \log \sigma(-sk-)$$

Trong đó σ(x)=1/(1+e-x) là sigmoid.

Logit | Mục tiêu | Loss đạt thấp khi

Positive s+ | σ(s+) → 1 | s+ lớn, source và context dương có dot product cao

Negative s- | σ(s-) → 0 | s- âm hoặc đủ nhỏ

Đây là binary logistic objective: positive pair được coi là nhãn 1, negative pair là nhãn 0. So với softmax toàn bộ vocabulary, negative sampling chỉ tính trên một số ít node negative nên rẻ hơn khi graph lớn.

9. Lan truyền ngược và cập nhật

9.1. Vòng lặp huấn luyện

```
for sources, positive_contexts, negative_contexts in train_loader:
    sources = sources.to(device)
    positive_contexts = positive_contexts.to(device)
    negative_contexts = negative_contexts.to(device)

    optimizer.zero_grad()
    positive_logits = model(sources, positive_contexts)
    negative_logits = model(sources.unsqueeze(1), negative_contexts)
    loss = criterion(positive_logits, negative_logits)
    loss.backward()
    optimizer.step()
```

- Forward: tra vector từ hai bảng embedding và tính dot product.
- Loss: so sánh positive score với negative scores.
- Backward: autograd tính gradient của loss theo các embedding đã được tra.
- Step: Adam cập nhật tham số bằng learning rate lr.
- Zero: xóa gradient cũ trước batch tiếp theo.

9.2. Gradient của một positive pair

Với s+ = u·v+ và sk- = u·vk-, đạo hàm theo các logit là:

$$\partial L/\partial s+ = \sigma(s+) - 1 \quad \partial L/\partial sk- = \sigma(sk-)$$

9.2.1. Suy ra đạo hàm từng bước

Đặt loss của một source cùng một positive context và K negative contexts là:

$$L = -\log \sigma(s+) - \Sigma k=1..K \log \sigma(-sk-)$$

Bước 1: đạo hàm của sigmoid. Với σ(x) = 1/(1+e−x):

$$\sigma'(x) = \sigma(x)(1-\sigma(x))$$

Từ đó có hai đồng nhất thức tiện dùng:

$$d \log \sigma(x)/dx = 1-\sigma(x) \quad 1-\sigma(-x) = \sigma(x)$$

Bước 2: đạo hàm theo positive logit.

$$\partial[-\log \sigma(s+)]/\partial s+ = -(1-\sigma(s+)) = \sigma(s+)-1$$

Nếu positive score còn nhỏ, σ(s+)−1 âm lớn về độ lớn, nên gradient descent sẽ làm score tăng.

Bước 3: đạo hàm theo negative logit. Thành phần thứ k là −log σ(−sk-). Quy tắc chain rule cho thêm đạo hàm của −sk-:

$$\partial[-\log \sigma(-sk-)]/\partial sk- = 1-\sigma(-sk-) = \sigma(sk-)$$

Vì vậy negative score có gradient dương. Gradient descent sẽ làm negative score giảm.

9.2.2. Từ logit sang các vector embedding

Logit là tích vô hướng:

$$s+ = uTv+ \quad sk- = uTvk-$$

Vì ∂(uTv)/∂u = v và ∂(uTv)/∂v = u, áp dụng chain rule ta có:

Suy ra gradient quan trọng:

$$\partial L/\partial u = (\sigma(s+)-1)v+ + \Sigma k \, \sigma(sk-)vk-$$

$$\partial L/\partial v+ = (\sigma(s+)-1)u \quad \partial L/\partial vk- = \sigma(sk-)u$$

Positive pair | $\sigma(s+)-1 < 0$. Khi trừ gradient, u và v+ dịch theo hướng làm u·v+ tăng.

Negative pair | $\sigma(sk-) > 0$. Khi trừ gradient, u và vk- dịch theo hướng làm u·vk- giảm.

Batch mean | Code gọi .mean(), nên tổng gradient của từng sample còn được chia theo số sample trong batch.

9.2.3. Ví dụ số một chiều

Để nhìn rõ dấu của gradient, giả sử chỉ có một chiều, một positive và một negative:

```
u = 0.5,    v+ = 0.8,    v- = -0.2
s+ = u*v+ = 0.4
s- = u*v- = -0.1
σ(s+) ≈ 0.5987,    σ(s-) ≈ 0.4750
```

Positive logit | $\partial L/\partial s+ = 0.5987-1 = -0.4013$

Negative logit | $\partial L/\partial s- = 0.4750$

Source vector | $\partial L/\partial u = (-0.4013)(0.8) + (0.4750)(-0.2) \approx -0.4160$

Positive vector | $\partial L/\partial v+ = (-0.4013)(0.5) \approx -0.2007$

Negative vector | $\partial L/\partial v- = (0.4750)(0.5) \approx 0.2375$

Nếu minh họa bằng SGD với $\eta=0.1$ thì u tăng từ 0.5 lên khoảng 0.5416, v+ tăng lên khoảng 0.8201, còn v- giảm xuống khoảng −0.2238. Khi đó positive dot product tăng và negative dot product giảm. Code thực tế dùng Adam nên độ lớn bước cập nhật không đúng bằng ví dụ SGD này, nhưng hướng tác động vẫn như vậy.

Trực giác:

• Source vector được kéo gần context dương.

• Source vector bị đẩy xa các context âm.

• Chỉ các row embedding xuất hiện trong batch nhận gradient trực tiếp.

9.3. Adam cập nhật tham số

Code dùng:

```
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
```

Adam giữ các thống kê chuyển động của gradient theo từng tham số, kết hợp momentum và chuẩn hóa theo độ lớn gradient. Với mỗi row embedding có gradient g, Adam tạo bước cập nhật thích nghi thay vì dùng một bước cố định đơn giản. Sau optimizer.step(), hai bảng source_embedding.weight và context_embedding.weight đều thay đổi.

10. Embedding đầu ra

```
def get_embeddings(self):
    return self.source_embedding.weight
```

fit() trả:

```
self.model.get_embeddings().detach().cpu().numpy()
```

Vì vậy output có shape:

(number_of_nodes, embedding_dimension)

Dòng thứ i tương ứng với node ở vị trí thứ i trong list(G.nodes()). Trong project, karate_node_ids.npy được lưu để bảo toàn mapping này khi đánh giá.

Code chỉ trả về source embedding. Một số biến thể word2vec dùng (source + context) / 2 hoặc trả context embedding; đó không phải hành vi hiện tại của project.

11. Pseudocode toàn bộ thuật toán

```
Input: graph G, p, q, dimension d, walks_per_node r,
       walk_length L, context_size w, negatives K

1. Preprocess:
   for each undirected edge (u, v):
       create transition state (u, v) and (v, u)
       for each candidate x in neighbors[current]:
           alpha = 1/p if x == previous
                   1   if x in neighbors[previous]
                   1/q otherwise
           prob[x] = edge_weight(current, x) * alpha
       normalize prob

2. Generate walks:
   repeat r times:
       for each node s in G:
           walk = [s]
           repeat L times:
               sample next node from weighted transition distribution
               append next node
           save walk

3. Positive samples:
   for each walk and each position i:
       source = walk[i]
       for j = i+1 ... min(i+w, len(walk)-1):
           save (source, walk[j])

4. Negative samples:
   count node frequencies in all walks
   make noise probabilities count^0.75 / sum(count^0.75)
   for each positive pair (s, c):
       remove s and c from candidates
       draw K negative contexts

5. Dataset:
   map node labels to integer IDs
   make tensors source[N], positive[N], negative[N,K]
   create TensorDataset and shuffled DataLoader

6. Train Skip-gram:
   score positive and negative dot products
   compute negative-sampling loss
   backward()
   Adam step()

7. Output:
   return source_embedding.weight as node embeddings.
```

12. Những điểm cần nhớ khi đọc implementation

Điểm | Hành vi hiện tại | Hệ quả

alias_edges | Cache candidates và probabilities; chưa phải Vose alias table | Đúng phân phối nhưng chưa tối ưu sampling kiểu alias

Độ dài walk | walk_length vòng lặp sau node bắt đầu | Danh sách dài walk_length + 1

Positive context | Chỉ lấy node ở phía sau source | Cặp có hướng và ít hơn cửa sổ hai phía

Sample trùng | Không deduplicate | Tần suất đồng xuất hiện được giữ lại

Negative sampling | Lấy với replace=True | Negative trong cùng nhóm có thể lặp

Negative “thật sự âm” | Chỉ loại source và positive context hiện tại | Có thể chọn node là positive context ở pair khác

Embedding trả về | source_embedding.weight | Không phải context table và không phải trung bình hai table

Đánh giá | Logistic regression trên node labels | Kiểm tra embedding phục vụ downstream classification

13. Kết luận

Thuật toán trong project tạo embedding theo chuỗi nguyên nhân sau:

Random walk quyết định node nào được xem là có ngữ cảnh tương tự. Skip-gram biến các quan sát đó thành bài toán phân loại nhị phân bằng dot product và negative sampling. Backpropagation cùng Adam lặp lại quá trình này cho đến khi source vectors mã hóa được cấu trúc mà các walk đã quan sát.

Tài liệu được sinh theo mã nguồn hiện tại của repository. Khi thay đổi cách lấy context, số bước walk, cách sample negative hoặc cách lấy embedding cuối, các phần tương ứng trong tài liệu cần được cập nhật.